

Desing Assignment

Design Assignment Instructions

1) Context (must be included in your analysis)

UnoArena: Global Real-Time Uno Platform & Massive Tournaments

The Problem: Build the backend for a highly competitive Uno platform that supports both individual, ad-hoc game rooms (2-10 players) and massive, multi-tiered elimination tournaments (up to 1,000,000 players). The system must handle split-second game mechanics (like calling "Uno!"), real-time tournament progression, and a global Elo-based ranking system.

Uno rules reference: <https://www.youtube.com/watch?v=rC-DYC3ZELM>
(<https://www.youtube.com/watch?v=rC-DYC3ZELM>)

Key domain rules for Uno! call mechanics:

- A player who plays their second-to-last card must call "Uno!" before the next player takes their turn.
- Any opponent may challenge the call within a 5-second window after the card is played.
- If a player fails to call "Uno!" and is successfully challenged, they draw 2 penalty cards.
- If a player is challenged but did correctly call "Uno!", the challenger draws 2 penalty cards.
- A challenge window closes as soon as the next player begins their turn.

Core Engineering Challenges:

- **Hierarchical Lifecycle Management (Rooms & Tournaments):** Managing lifecycle state transitions at two levels. Individual rooms move through clear states (waiting -> in_progress -> completed). At tournament level, rounds must coordinate up to 1,000,000 players and absorb the first-round surge of over 100,000 simultaneous matches. Room completion must securely trigger winner advancement and next-round progression.
- **Authoritative Deck & RNG Service:** All card shuffling and draws are generated server-side. Every single state change (e.g., card.played, color.changed, penalty.drawn) is

appended to an immutable game log before being broadcast, making every random outcome deterministic, auditable, and replay-safe. This is critical for dispute resolution in high-stakes tiers.

- **Real-Time Updates & Spectator Privacy:** Player actions and game-state updates must be propagated in near real time to players and observers at very large scale. Observers can watch rooms, but they must only see public information (player names and discard stack), never private hands. The system must preserve responsiveness under heavy connection load without compromising gameplay consistency.
- **Strict Concurrency & Reactive Rules Enforcement:** The room-state service must serialize concurrent REST requests with strict sequence numbers. Stale actions are rejected with HTTP 409 Conflict, and clients reconcile via SSE.
- **Disconnection Handling & Session Continuity:** The platform must handle temporary and permanent disconnections according to the following domain rules:
 - A disconnected player has a **60-second reconnection window** before being considered inactive.
 - During the reconnection window, the disconnected player's turn is skipped (as if they passed), and no bot substitution occurs.
 - If the window expires during the player's turn, an automatic forfeit is issued.
 - Forfeit in a casual room ends the player's participation; the game continues with remaining players.
 - Forfeit in a tournament room counts as a loss for that match; the player is eliminated from the tournament.
 - A player who reconnects within the window resumes with their original hand intact.
- **Round-Based Tournament Progression Rules:** Tournaments proceed in elimination rounds until 10 or fewer players remain, at which point a final room is created. Each round, players are distributed into rooms of up to 10 players. Within a room, players play a **best-of-three series** (a "match" = up to 3 individual games). The **top 3 players** by match wins advance; in case of a tie, the player with the lower cumulative card-point total in the tied games advances. If still tied, advancement order is determined by earliest time of final-game completion.
- **Security Hardening, Session Control & Rate Limiting:** Include strong auth boundaries, input validation, signed event integrity where needed, and audit logs for sensitive operations. Enforce single-active-session per player (new login invalidates old session). Apply multi-layer rate limits (per IP, per user, per room/tournament action) with adaptive throttling.
- **Tournament Analytics & Bracket Read Models:** The platform must absorb spikes of game.completed events and update read-optimized views for player stats and bracket visualization.
- **Elo & Ranking Rules:** The global Elo-based ranking applies to casual (ad-hoc) rooms only. Tournament play uses a separate tournament-placement rating. Elo is updated once per completed game (not per match or per tournament). The Elo delta is calculated from final placement order within the room (1st through last). Abandoned games (forfeit by all remaining players) do not affect Elo.

What makes it hard at 1,000,000 users: Massive synchronization of tournament rounds while keeping gameplay responsive for very large numbers of connected players and observers.

2) Assignment objective

Produce a rigorous, behavior-complete domain model for the UnoArena problem using a Domain-Driven Design (DDD) approach, with a strong focus on behavior, rules, and business consistency under extreme concurrency and scale.

The assignment must explicitly analyze:

- Domain language and business rules.
- Boundaries and responsibilities in the domain.
- Full domain event model.
- Edge cases and failure paths (not only happy paths).

3) Scope constraints

- This checkpoint is about **domain design**, not infrastructure details.
- Do **not** go deep into low-level architecture, deployment, framework choices, cloud sizing, or protocol internals.
- The **client connection protocol** details are **out of scope for this iteration** and will be defined in the next one. You may, however, state explicit assumptions about connection semantics (e.g., "we assume at-least-once delivery") without designing the protocol itself. Any such assumptions must be listed in deliverable 8.

4) Mandatory methodology

Use **EventStorming** as the primary discovery and analysis technique.

At minimum, your EventStorming outcome must cover:

- Main business flows (room lifecycle, match lifecycle, tournament progression).
- Exceptional flows (timeouts, stale commands, disconnects, forfeits, invalid actions).
- Cross-context interactions triggered by domain events.
- Invariants and policy decisions that protect consistency.

5) Required deliverables

Your assignment submission must include:

1. Domain glossary

- Ubiquitous language with precise definitions.
- Must distinguish: *game* (single Uno game), *match* (best-of-three series within a room), *round* (one elimination tier in a tournament), *tournament*.

1. Bounded contexts and context map

- Proposed contexts (for example: Room Gameplay, Tournament Orchestration, Ranking, Identity/Session, Spectator View).
- Relationships between contexts (upstream/downstream or equivalent).
- Explicit treatment of the Spectator View context: what information crosses its boundary, what is withheld, and which domain events drive its updates.

1. Aggregates, entities, value objects

- Candidate aggregates and consistency boundaries.
- Key invariants each aggregate must enforce.

1. Commands and domain events catalog

- List core commands and resulting events.
- Include causality (what triggers what) and idempotency considerations.

1. Domain event flow narratives

- End-to-end event sequences for:
 - Room creation to completion.
 - Tournament round advancement.
 - Elo/ranking updates after game completion.
- Include both synchronous decision points and asynchronous propagation.

1. Edge cases and failure-path analysis

- At least the following categories:
 - Concurrent conflicting actions (e.g., two players simultaneously playing a card).
 - Disconnections and late rejoin attempts.
 - Stale commands and replayed commands.
 - Partial failures between contexts.
 - Security and abuse scenarios (session takeover, spam, flooding).
 - Spectator privacy violations (e.g., a player attempting to read another player's hand via the spectator channel).
- For each case, define expected domain behavior and emitted events.

1. Consistency and recovery strategy (domain level)

- Retries, deduplication, compensation/saga decisions at the business level.
- How invariant violations are prevented or detected.

1. Open questions and assumptions

- Clearly separate assumptions from validated requirements.
- Include any connection-semantics assumptions made under the scope constraints of Section 3.

6) Evaluation criteria

Submissions will be evaluated on:

- Correctness and completeness of domain modeling.
- Quality and coverage of domain event analysis.
- Depth of edge-case and failure-path reasoning.
- Clarity of bounded contexts and invariants.
- Consistency of ubiquitous language and EventStorming artifacts.
- Ability to keep architectural detail out of scope while preserving domain rigor.

7) Submission format and deadline

- The delivery must be provided as one or more Markdown files, organized at minimum into:
 - A root README.md or index.md that lists and links all documents.
 - One file per major deliverable (or clearly separated sections if combined).

- The submission may include supporting visual artifacts such as:
 - ASCII diagrams
 - Mermaid charts
 - PNG images
- Students must submit the repository link on time.
- The grading cutoff is based on git history: **the last commit with timestamp up to 23:59 (April 5th)** is the one that will be considered.